



BD01 : TP 1

(select ... from... where)

Objectifs :

- Exécuter un script SQL dans l'environnement **XAMPP** (et plus particulièrement **PhpMyAdmin**).
- Apprendre progressivement le langage SQL et la construction des requêtes SQL (cet apprentissage se fera tout au long des travaux pratiques)

Qu'est-ce qu'un fichier script ?

Un **script** est un fichier texte qui contient une **séquence d'instructions SQL**.

Ce fichier doit être ASCII pur. (Il ne peut pas provenir d'un traitement de texte qui injecte des caractères de mise en page).

Chaque instruction est séparée d'un point-virgule.

Si on souhaite indiquer un commentaire, on commence la ligne par deux tirets et un espace, exemple :

```
-- commentaire
```

Exécuter un fichier script produit le même effet qu'écrire et exécuter successivement chacune de ses instructions en mode commande.

Les erreurs présentes dans un script sont signalées à son exécution. **Seules les instructions qui ont généré ces erreurs**, ainsi que les suivantes ne sont pas exécutées. Si vous relancer dès lors le script après avoir corrigé ces erreurs, vous engendrez de nouvelles erreurs puisque vous tentez d'exécuter à nouveau les instructions qui avaient correctement été exécutées la première fois.

Comme exemple, voici le script de création et d'insertion de données d'une des bases de données qui vont nous servir pour nos laboratoires :

```
CREATE DATABASE IF NOT EXISTS `fournisseurs` DEFAULT CHARACTER SET latin1 COLLATE latin1_swedish_ci;  
USE `fournisseurs`;
```

```
DROP TABLE IF EXISTS `J`;  
CREATE TABLE IF NOT EXISTS `J` (  
  `ID_J` char(4) NOT NULL,  
  `JNAME` char(20) NOT NULL,  
  `CITY` char(10) NOT NULL,  
  PRIMARY KEY (`ID_J`)  
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

```
INSERT INTO `J` (`ID_J`, `JNAME`, `CITY`) VALUES  
( 'J1', 'Sorter', 'Paris'),  
( 'J2', 'Display', 'Rome'),  
( 'J3', 'OCR', 'Athens'),  
( 'J4', 'Console', 'Athens'),  
( 'J5', 'RAID', 'London'),  
( 'J6', 'EDS', 'Oslo'),  
( 'J7', 'Tape', 'London');
```



```
DROP TABLE IF EXISTS `P`;
CREATE TABLE IF NOT EXISTS `P` (
  `ID_P` char(4) NOT NULL,
  `PNAME` char(20) NOT NULL,
  `COLOR` char(8) NOT NULL,
  `WEIGHT` int(11) NOT NULL,
  `CITY` char(10) NOT NULL,
  PRIMARY KEY (`ID_P`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

INSERT INTO `P` (`ID_P`, `PNAME`, `COLOR`, `WEIGHT`, `CITY`) VALUES
('P1', 'Nut', 'Red', 12, 'London'),
('P2', 'Bolt', 'Green', 17, 'Paris'),
('P3', 'Screw', 'Blue', 17, 'Rome'),
('P4', 'Screw', 'Red', 14, 'London'),
('P5', 'Cam', 'Blue', 12, 'Paris'),
('P6', 'Cog', 'Red', 19, 'London');

DROP TABLE IF EXISTS `S`;
CREATE TABLE IF NOT EXISTS `S` (
  `ID_S` char(4) NOT NULL,
  `SNAME` char(20) NOT NULL,
  `STATUS` char(2) NOT NULL,
  `CITY` char(10) NOT NULL,
  PRIMARY KEY (`ID_S`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

INSERT INTO `S` (`ID_S`, `SNAME`, `STATUS`, `CITY`) VALUES
('S1', 'Smith', '20', 'London'),
('S2', 'Jones', '10', 'Paris'),
('S3', 'Blake', '30', 'Paris'),
('S4', 'Clark', '20', 'London'),
('S5', 'Adams', '30', 'Athens');

DROP TABLE IF EXISTS `SPJ`;
CREATE TABLE IF NOT EXISTS `SPJ` (
  `ID_S` char(4) NOT NULL,
  `ID_P` char(4) NOT NULL,
  `ID_J` char(4) NOT NULL,
  `QTY` int(11) NOT NULL,
  `DATE_DERNIERE_LIVRAISON` date NOT NULL,
  PRIMARY KEY (`ID_S`, `ID_P`, `ID_J`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

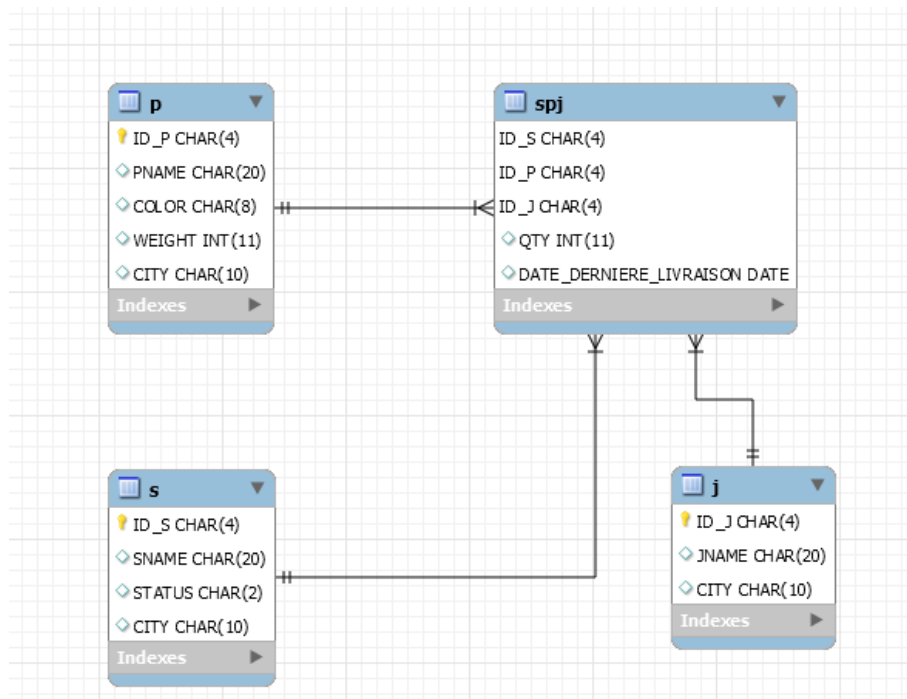
INSERT INTO `SPJ` (`ID_S`, `ID_P`, `ID_J`, `QTY`, `DATE_DERNIERE_LIVRAISON`) VALUES
('S1', 'P1', 'J1', 200, '2020-10-05'),
('S1', 'P1', 'J4', 700, '2021-05-10'),
('S2', 'P3', 'J1', 400, '2021-05-20'),
('S2', 'P3', 'J2', 200, '2020-07-30'),
('S2', 'P3', 'J3', 200, '2010-05-10'),
('S2', 'P3', 'J4', 500, '2021-10-03'),
('S2', 'P3', 'J5', 600, '2020-09-20'),
('S2', 'P3', 'J6', 400, '2022-05-12'),
('S2', 'P3', 'J7', 800, '2022-08-23'),
('S2', 'P5', 'J2', 100, '2022-06-23'),
('S3', 'P3', 'J1', 200, '2019-07-07'),
('S3', 'P4', 'J2', 500, '2022-05-18'),
('S4', 'P6', 'J3', 300, '2021-05-10'),
('S4', 'P6', 'J7', 300, '2020-09-16'),
('S5', 'P1', 'J4', 100, '2022-03-18'),
('S5', 'P2', 'J2', 200, '2021-11-10'),
('S5', 'P2', 'J4', 100, '2022-04-17'),
('S5', 'P3', 'J4', 200, '2015-05-19'),
('S5', 'P4', 'J4', 800, '2016-05-10'),
('S5', 'P5', 'J4', 400, '2018-12-16'),
('S5', 'P5', 'J5', 500, '2019-02-08'),
('S5', 'P5', 'J7', 100, '2020-06-25'),
('S5', 'P6', 'J2', 200, '2019-02-09'),
('S5', 'P6', 'J4', 500, '2018-10-10');
```



```
ALTER TABLE `SPJ`  
  ADD CONSTRAINT `pk_SPJ_S` FOREIGN KEY (`ID_S`) REFERENCES `S` (`ID_S`),  
  ADD CONSTRAINT `pk_SPJ_P` FOREIGN KEY (`ID_P`) REFERENCES `P` (`ID_P`),  
  ADD CONSTRAINT `pk_SPJ_J` FOREIGN KEY (`ID_J`) REFERENCES `J` (`ID_J`);
```

Modèle relationnel

Voici une représentation graphique des différentes tables



Exécution de ce fichier script

Travail 1

Ce premier travail consiste à créer la base de données qui servira pour beaucoup de vos travaux pratiques à l'aide du script `spj_mysql.sql`.

Pour ce faire :

1. Lancez XAMPP
2. Lancez les serveurs Apache et MySQL
3. Ouvrez PhpMyAdmin (en cliquant sur le bouton Admin) et rendez-vous à l'onglet « Import ».
4. Appuyez sur le bouton Choisir un fichier et parcourez le disque pour choisir le fichier `spj_mysql.sql`
5. Tout en bas de l'onglet, appuyez sur Go
6. Rafraîchissez l'onglet de gauche en appuyant sur la flèche verte et vous devriez voir la base de données '**fournisseurs**' apparaître.



Si l'exécution du script ne se déroule pas correctement, un message comportant les erreurs s'affiche. Ces messages sont souvent peu explicites mais guident quand même un peu la correction de l'erreur.

Pour vérifier que tout s'est bien passé, choisissez l'onglet `SQL` qui vous permet d'exécuter toutes vos requêtes, et exécutez la requête suivante :

```
SELECT * FROM S
```

Vous devriez voir apparaître les résultats suivants :

ID_S	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

La suite du travail se présente sous forme de tutoriel.

Vous êtes invité à lire les informations données et à taper les instructions mises en *italique*. N'oubliez pas d'exécuter chaque instruction une par une.

Syntaxe de base (la plus simple) pour la commande **SELECT** :

Remarque : les mots clés peuvent être écrits en majuscule, minuscule ou n'importe quelle combinaison (en d'autres termes, SQL ne fait pas attention à la casse en ce qui concerne les mots clés et les noms des champs). Nous mettons les mots clés en majuscule dans les explications de la syntaxe et les premiers exemples, mais, par la suite, nous lèverons cette contrainte.

```
SELECT liste_champs_à_afficher FROM nom_de_table
```

Explications sommaires :

- *liste_champs_à_afficher* : on appelle cette partie la **projection**. Elle indique tous les champs qui devront apparaître dans le résultat de la requête
- *nom_de_table* : nom de la table d'où proviennent les données. On verra bientôt qu'il peut y avoir plus qu'une seule table (ce sera d'ailleurs le cas le plus fréquent !)

Remarque : le mot clé **DISTINCT** permet de garder, dans le résultat, uniquement des lignes distinctes par la valeur d'un des champs au moins. Quand faut-il l'utiliser ?

Lorsque les deux conditions ci-dessous sont remplies

1. Si on veut obtenir un résultat qui est sans doublon, quel que soit le jeu de données. Dans les exercices, ce sera dit explicitement dans l'énoncé.
2. La projection choisie (la liste de champs) ne contient pas le (ou les) champ(s) identifiants. Dans ces premiers exemples où l'on travaille avec une seule table, cette situation apparaît quand la projection ne contient pas la clé primaire de la table (dans la suite du cours, d'autres cas plus complexes seront envisagés).

Attention, toutefois, à ne pas utiliser ce mot clé quand ce n'est pas nécessaire. En effet, il oblige le système à vérifier qu'il n'y a aucun doublon dans le résultat, ce qui, potentiellement, peut nuire à la performance (s'il y a beaucoup de lignes dans le résultat).

Exemples :

1. Afficher la colonne `sname` de la table `s`
`SELECT sname FROM s`

2. Afficher la colonne `sname` de la table `s`, **sans doublon**
`SELECT DISTINCT sname FROM s`

Explication : la projection ne contenant pas la clé primaire (`ID_S`), le résultat pourrait contenir des doublons.

3. Afficher les colonnes `sname` et `city` de la table `s`
`SELECT sname, city FROM s`

4. Afficher les colonnes `sname` et `city` de la table `s`, **sans doublon**
`SELECT DISTINCT sname, city FROM s`

5. Afficher la colonne `sname` en l'appelant `nom`
`SELECT sname nom FROM s` ou
`SELECT sname AS nom FROM s`



Travail 2

Imaginez ce que l'instruction affichera et décrivez-le en quelques mots. **Ensuite**, exécutez la requête pour vérifier.

1. `SELECT * FROM J`

2. `SELECT pname city FROM P`

3. `SELECT DISTINCT color couleur FROM P`

Sélectionner des données particulières dans une table

```
SELECT liste_champs_à_afficher  
FROM nom_de_table  
WHERE critères_de_sélection
```

La clause **where** exprime la condition de sélection (restriction) à effectuer sur le résultat obtenu de `select... from`.

Principe :

1. On construit la table correspondante à `select ... from`
2. On ne conserve que les lignes pour lesquelles l'évaluation de la condition est vraie.

En réalité, l'instruction SQL sera optimisée par le système. Rien ne dit que le travail s'effectuera vraiment de cette manière. La seule certitude est que le résultat serait semblable si on effectuait le travail de cette manière.

La condition peut être une condition composée s'exprimant entre autres à l'aide des opérateurs `=`, `>`, `>=`, `<`, `<=`, `<>`, `AND`, `OR`, `NOT`, `BETWEEN`, `IN`, `ANY`, `EXISTS` (`IN`, `ANY`, `EXISTS` seront vu plus tard)



Travail 3

Essayez les exemples suivants :

- 1) Le nom des fournisseurs dont le statut est "20".
`select sname from S where status = 20`
- 2) Les fournisseurs de Paris dont le statut est strictement supérieur à 20.
`select * from s where status > 20 and city = 'Paris'`
- 3) Les produits rouges ou verts dont le poids est strictement supérieur à 15
`select * from P where (color='Red' or color='Green') and weight > 15`
- 4) Pour les livraisons dont la quantité de produits livrés est entre 500 et 700, donnez les identifiants de produit ainsi que la quantité.
`select id_p, qty from spj where qty >= 500 and qty <=700`
ou
`select id_p, qty from spj where qty between 500 and 700`
- 5) Idem pour les livraisons dont la quantité de produits livrés n'est pas entre 500 et 700
`select id_p,qty from spj where qty < 500 or qty >700`
ou
`select id_p,qty from spj where qty not between 500 and 700`
- 6) Le nom et la couleur des produits dont le nom est "Bolt" ou "Cam"
`select pname, color from P where pname='Bolt' or pname='Cam'`
ou
`select pname, color from P where pname in ('Bolt', 'Cam')`



EXERCICES

Traduisez en SQL les requêtes suivantes. Pour chaque requête, on vous donne le résultat attendu avec les données existantes.

1) On veut afficher le contenu de la table des fournisseurs.

ID_S	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

2) On veut le nom et la ville de tous les fournisseurs avec comme intitulé NOM et VILLE

NOM	VILLE
Smith	London
Jones	Paris
Blake	Paris
Clark	London
Adams	Athens

3) On veut le nom des fournisseurs dont la ville est London ou Paris

SNAME
Smith
Jones
Blake
Clark

4) On veut le nom des fournisseurs dont le statut vaut strictement moins de 25 et qui sont de Paris.

SNAME
Jones

5) On veut obtenir le nom des fournisseurs dont le statut **n'est pas** dans l'intervalle fermé [15,25]. Autrement dit, le statut doit être strictement inférieur à 15 ou strictement supérieur à 25.

SNAME
Jones
Blake
Adams

6) On veut obtenir les noms des pièces rouges ou bleues. On ne veut pas de doublon.

PNAME
Nut
Screw
Cam
Cog